

Feature Engineering & Exploratory Data Analysis (EDA)

This notebook produces the final modeling dataset.

All downstream modeling notebooks should load:

final_engineered_claims_dataset.csv

Import Libraries

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
```

Load Data

```
# Load datasets
df = pd.read_csv("cleaned_final_claims_dataset.csv")
```

```
df.head()
```

```
.dataframe tbody tr th {
    vertical-align: top;
}

.dataframe thead th {
    text-align: right;
}
```

	Claim_ID	Policy_ID	Accident_Date	FNOL_Date	Claim_Type	Claim_Complexity	Fraud_Flag	Litigation_Flag	Estimated_Claim_Amount	Unlabeled
0	CLM30000	POL14506	2019-12-19	2019-12-19	Theft	Medium	False	True	5243	2
1	CLM30001	POL14338	2018-12-30	2018-12-31	Collision	Low	False	False	3934	2
2	CLM30002	POL13575	2021-10-19	2021-10-19	Other	Medium	False	False	153631	1
3	CLM30003	POL10138	2021-06-18	2021-06-18	Weather	Low	False	False	2812	1
4	CLM30004	POL12316	2021-03-21	2021-03-24	Theft	Low	False	False	5094	4

5 rows × 30 columns

```
df.shape
```

```
(8000, 30)
```

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 8000 entries, 0 to 7999
Data columns (total 30 columns):
#   Column              Non-Null Count  Dtype
---  ---
0   Claim_ID            8000 non-null   object
1   Policy_ID           8000 non-null   object
2   Accident_Date       8000 non-null   object
```

```

3  FNOL_Date                8000 non-null  object
4  Claim_Type               8000 non-null  object
5  Claim_Complexity         8000 non-null  object
6  Fraud_Flag               8000 non-null  bool
7  Litigation_Flag         8000 non-null  bool
8  Estimated_Claim_Amount   8000 non-null  int64
9  Ultimate_Claim_Amount    7575 non-null  float64
10 Severity_Band            8000 non-null  object
11 Settlement_Date          7575 non-null  object
12 Status                   8000 non-null  object
13 Days_To_FNOL             8000 non-null  int64
14 Ultimate_Claim_Amount_Log 7575 non-null  float64
15 Customer_ID              8000 non-null  object
16 Age_of_Driver            8000 non-null  int64
17 Gender                   8000 non-null  object
18 Occupation               8000 non-null  object
19 Region                   8000 non-null  object
20 Annual_Mileage           8000 non-null  int64
21 Driving_Experience_Years 8000 non-null  int64
22 Vehicle_Type             8000 non-null  object
23 Vehicle_Age              8000 non-null  int64
24 Credit_Score_Band        8000 non-null  object
25 Num_Third_Parties        8000 non-null  float64
26 Max_ThirdParty_Severity  8000 non-null  float64
27 Any_ThirdParty_Injury    8000 non-null  float64
28 Num_Passengers           8000 non-null  float64
29 Num_Drivers              8000 non-null  float64
dtypes: bool(2), float64(7), int64(6), object(15)
memory usage: 1.7+ MB

```

```

# Convert date columns
date_cols = ["Accident_Date", "FNOL_Date", "Settlement_Date"]
for col in date_cols:
    if col in df.columns:
        df[col] = pd.to_datetime(df[col], errors="coerce")

```

```

# Convert categorical columns from object datatype to category for memory optimisation
cat_cols = ["Claim_Type", "Claim_Complexity", "Severity_Band", "Status", "Gender", "Occupation", "Region", "Vehicle_Type",
"Credit_Score_Band"]
for col in cat_cols:
    if col in df.columns:
        df[col] = df[col].astype("category")

```

```

# Check the results
df.info()

```

```

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 8000 entries, 0 to 7999
Data columns (total 30 columns):
#   Column                Non-Null Count  Dtype
---  -
0   Claim_ID              8000 non-null  object
1   Policy_ID             8000 non-null  object
2   Accident_Date         8000 non-null  datetime64[ns]
3   FNOL_Date             8000 non-null  datetime64[ns]
4   Claim_Type            8000 non-null  category
5   Claim_Complexity      8000 non-null  category
6   Fraud_Flag            8000 non-null  bool
7   Litigation_Flag       8000 non-null  bool
8   Estimated_Claim_Amount 8000 non-null  int64
9   Ultimate_Claim_Amount 7575 non-null  float64
10  Severity_Band         8000 non-null  category
11  Settlement_Date        7575 non-null  datetime64[ns]
12  Status                 8000 non-null  category
13  Days_To_FNOL          8000 non-null  int64
14  Ultimate_Claim_Amount_Log 7575 non-null  float64
15  Customer_ID           8000 non-null  object
16  Age_of_Driver         8000 non-null  int64
17  Gender                 8000 non-null  category

```

```

18 Occupation          8000 non-null  category
19 Region              8000 non-null  category
20 Annual_Mileage       8000 non-null  int64
21 Driving_Experience_Years 8000 non-null  int64
22 Vehicle_Type         8000 non-null  category
23 Vehicle_Age          8000 non-null  int64
24 Credit_Score_Band     8000 non-null  category
25 Num_Third_Parties     8000 non-null  float64
26 Max_ThirdParty_Severity 8000 non-null  float64
27 Any_ThirdParty_Injury  8000 non-null  float64
28 Num_Passengers        8000 non-null  float64
29 Num_Drivers           8000 non-null  float64
dtypes: bool(2), category(9), datetime64[ns](3), float64(7), int64(6), object(3)
memory usage: 1.2+ MB

```

Feature Engineering

Engineer some features into the existing variables to understand patterns and behaviour

Time based features

```

# Days to FNOL- How quickly claims are reported.It measures reporting delay.
# Number of days between Accident date & FNOL- This feature helps to estimate days interval
df["Days_To_FNOL"] = (df["FNOL_Date"] - df["Accident_Date"]).dt.days

```

```

# Days FNOL_to_Settlement- Number of days between first notification of loss to settlement
# This feature helps to estimate how long it takes for settlement to be made and ideal for measuring customer satisfaction.
df["Days_FNOL_to_Settlement"] = (pd.to_datetime(df["Settlement_Date"]) - pd.to_datetime(df["FNOL_Date"])).dt.days

```

Vehicle Risk

```

# Vehicle_Age - The older the car, the higher the maintenance issues and the risk score
df["Vehicle_Age_Risk"] = pd.cut(df["Vehicle_Age"],bins=[-1, 3, 7, 15, 100],labels=[1, 2, 3, 4],include_lowest=True).astype("Int64")

```

```

# Vehicle Type Risk- Risk scores are assigned on the logic of rider/driver exposure, mileage, weight of car, and common insurance patterns.
# Certain vehicle types are riskier in accidents
vehicle_type_risk_map = {"Hatchback": 1,"Sedan": 2,"Coupe": 3,"SUV": 3, "Suv": 3, "Van": 4,"Motorcycle": 4}
df["Vehicle_Type_Risk"] = (df["Vehicle_Type"].map(vehicle_type_risk_map))

```

```

# Overall Vehicle Risk-Vehicle risk was engineered by combining vehicle age bands and vehicle type risk scores.
df["Vehicle_Risk"] = df["Vehicle_Age_Risk"] + df["Vehicle_Type_Risk"]

```

Driver Risk

```

# Driver Age Risk - Younger and elderly drivers are higher risk due to inexperience or slower reflexes
df["Driver_Age_Risk"] = pd.cut(df["Age_of_Driver"],bins=[17, 25, 40, 65, 100],labels=[4, 2, 1, 3]).astype(int)

```

```

# Driver Experience Risk- The longer the experience, the lower the risk and vice versa.
# Drivers with fewer years of experience are higher risk
df["Experience_Risk"] = pd.cut(df["Driving_Experience_Years"],bins=[-1, 2, 5, 15, 61],labels=[4, 3, 2, 1],include_lowest=True).astype("Int64")

```

```

# Overall Driver Risk- Driver risk engineered by combining driver's age bands and driver's experience risk scores.
df["Driver_Risk"] = df["Driver_Age_Risk"] + df["Experience_Risk"]

```

Season/ Weather-based Feature

```
# Season of Accident

# Extract month and year
df["Accident_Month"] = df["Accident_Date"].dt.month
df["Accident_Year"] = df["Accident_Date"].dt.year

# Map month to season number
df["Accident_Season_Num"] = df["Accident_Month"] % 12 // 3 + 1

# Map season number to season name
season_name_map = {1: "Winter", 2: "Spring", 3: "Summer", 4: "Fall"}
df["Season"] = df["Accident_Season_Num"].map(season_name_map)

# Map season name to risk score
season_risk_map = {
    "Winter": 3, # High risk
    "Spring": 2, # Moderate risk
    "Summer": 1, # Low risk
    "Fall": 2 # Moderate risk
}
df["Accident_Season_Risk"] = df["Season"].map(season_risk_map)

# Show first 10 rows of the new columns (fixed column names)
print(df[["Accident_Date", "Accident_Season_Num", "Season", "Accident_Season_Risk"]].head(10))

# Check counts by season
print(df["Season"].value_counts())

# Check frequency of each risk score
print(df["Accident_Season_Risk"].value_counts())
```

```

Accident_Date  Accident_Season_Num  Season  Accident_Season_Risk
0    2019-12-19                1  Winter                3
1    2018-12-30                1  Winter                3
2    2021-10-19                4   Fall                2
3    2021-06-18                3  Summer                1
4    2021-03-21                2  Spring                2
5    2020-04-12                2  Spring                2
6    2019-11-12                4   Fall                2
7    2024-10-30                4   Fall                2
8    2019-09-10                4   Fall                2
9    2025-05-03                2  Spring                2
Season
Spring    2091
Fall      2014
Winter    1962
Summer    1933
Name: count, dtype: int64
Accident_Season_Risk
2    4105
3    1962
1    1933
Name: count, dtype: int64
```

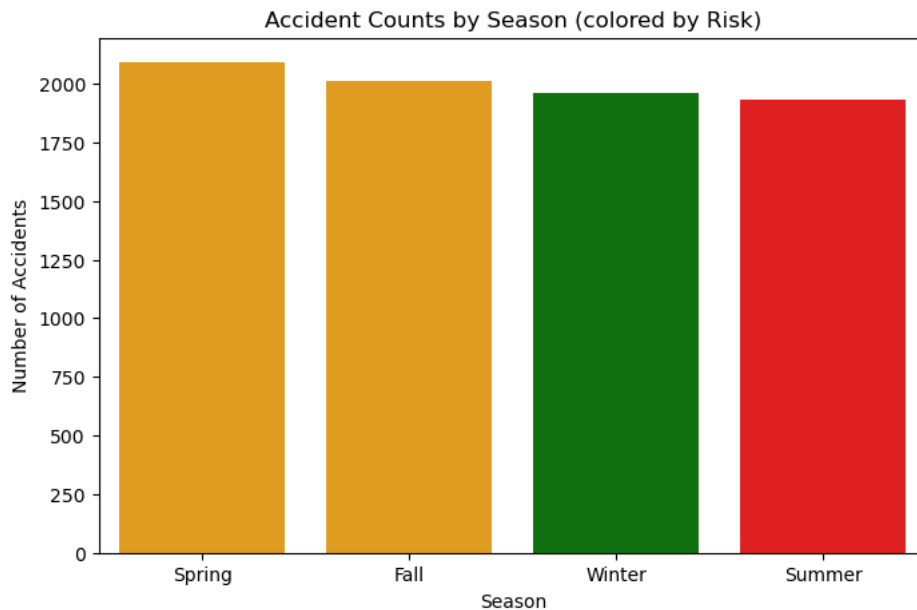
```
# Define a color map for risk
risk_colors = {1: "green", 2: "orange", 3: "red"}

plt.figure(figsize=(8,5))
sns.barplot(
    x=df["Season"].value_counts().index,
    y=df["Season"].value_counts().values,
    palette=[risk_colors[r] for r in df.groupby("Season")["Accident_Season_Risk"].first()]
)
plt.xlabel("Season")
plt.ylabel("Number of Accidents")
plt.title("Accident Counts by Season (colored by Risk)")
plt.show()
```

C:\Users\adeol\AppData\Local\Temp\ipykernel_51600\2101178910.py:5: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `x` variable to `hue` and set `legend=False` for the same effect.

```
sns.barplot(
```



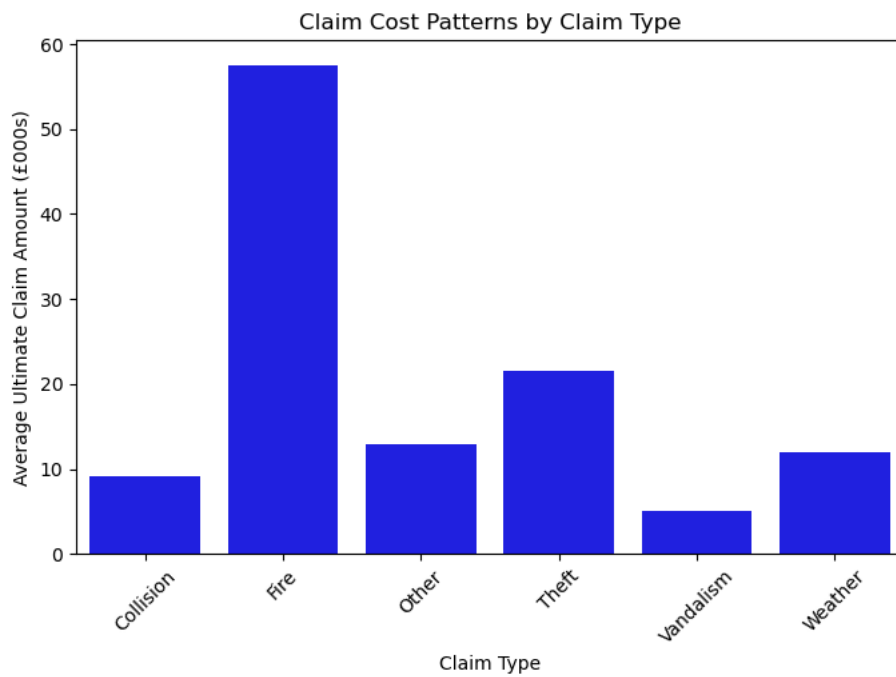
Claim Type Analysis

```
# Aggregate mean claim amount by Claim_Type
claim_cost_by_type = (df.groupby("Claim_Type", observed=True)["Ultimate_Claim_Amount"].mean().sort_values(ascending=False))

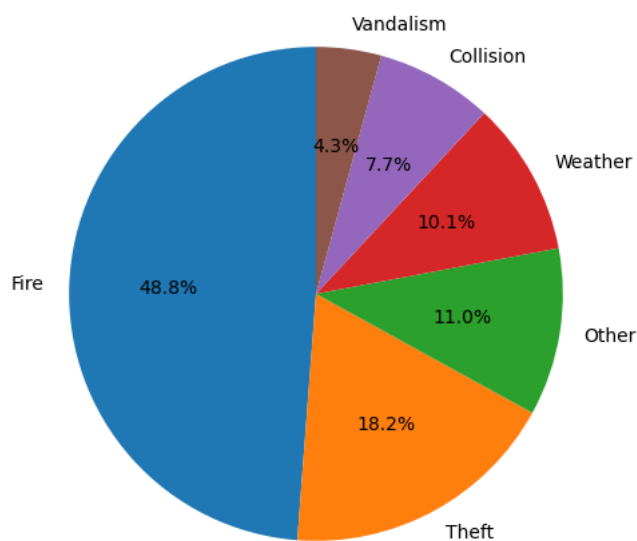
# Convert to thousands
claim_cost_by_type_k = claim_cost_by_type / 1000

# Plot bar chart
plt.figure(figsize=(8,5))
sns.barplot(x=claim_cost_by_type_k.index, y=claim_cost_by_type_k.values, color="blue")
plt.title("Claim Cost Patterns by Claim Type")
plt.ylabel("Average Ultimate Claim Amount (£000s)")
plt.xlabel("Claim Type")
plt.xticks(rotation=45)
plt.show()

# : Pie chart
plt.figure(figsize=(6,6))
claim_cost_by_type_k.plot(kind="pie", autopct='%1.1f%%', startangle=90)
plt.ylabel("") # Hide y-label
plt.title("Proportion of Average Claim Amount by Claim Type")
plt.show()
```



Proportion of Average Claim Amount by Claim Type



Claim_Type One-Hot Encoding

```
# One-hot encoding Claim_Type to convert categorical claims categories into numeric binary features for machine learning.
# Collision is dropped as a reference category to avoid multicollinearity

# One-hot encode Claim_Type
claim_type_dummies = pd.get_dummies(df["Claim_Type"], prefix="Claim_Type", drop_first=True)

# Add to dataset-Joins the new dummy columns with the original dataset.
df = pd.concat([df, claim_type_dummies], axis=1)

# Drop original categorical column before modeling- Claim Type is deleted as ML models cannot use strings
df = df.drop(columns=["Claim_Type"])
```

Gender Encoding

```
# Encode Gender as a binary feature for modeling
# Male = 1, Female = 0
df["Gender_Code"] = df["Gender"].map({"Male": 1, "Female": 0})

# Convert Gender_Code datatype from category to numeric
```

```
if df['Gender_Code'].dtype.name == 'category':
    df['Gender_Code'] = df['Gender_Code'].cat.codes.astype('int64')

# Drop original categorical Gender column-Gender is deleted as ML models cannot use strings
df = df.drop(columns=["Gender"])
```

Review Final Output

```
# Inspect dataset after feature engineering and column drops
# to verify structure, data types, and engineered features.
df.info()
df.columns
df.head()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 8000 entries, 0 to 7999
Data columns (total 46 columns):
#   Column                                Non-Null Count  Dtype
---  -
0   Claim_ID                             8000 non-null   object
1   Policy_ID                             8000 non-null   object
2   Accident_Date                         8000 non-null   datetime64[ns]
3   FNOL_Date                             8000 non-null   datetime64[ns]
4   Claim_Complexity                       8000 non-null   category
5   Fraud_Flag                             8000 non-null   bool
6   Litigation_Flag                       8000 non-null   bool
7   Estimated_Claim_Amount                 8000 non-null   int64
8   Ultimate_Claim_Amount                 7575 non-null   float64
9   Severity_Band                         8000 non-null   category
10  Settlement_Date                       7575 non-null   datetime64[ns]
11  Status                                 8000 non-null   category
12  Days_To_FNOL                           8000 non-null   int64
13  Ultimate_Claim_Amount_Log              7575 non-null   float64
14  Customer_ID                           8000 non-null   object
15  Age_of_Driver                         8000 non-null   int64
16  Occupation                             8000 non-null   category
17  Region                                 8000 non-null   category
18  Annual_Mileage                         8000 non-null   int64
19  Driving_Experience_Years               8000 non-null   int64
20  Vehicle_Type                           8000 non-null   category
21  Vehicle_Age                           8000 non-null   int64
22  Credit_Score_Band                     8000 non-null   category
23  Num_Third_Parties                     8000 non-null   float64
24  Max_ThirdParty_Severity                8000 non-null   float64
25  Any_ThirdParty_Injury                  8000 non-null   float64
26  Num_Passengers                         8000 non-null   float64
27  Num_Drivers                           8000 non-null   float64
28  Days_FNOL_to_Settlement               7575 non-null   float64
29  Vehicle_Age_Risk                       8000 non-null   int64
30  Vehicle_Type_Risk                     8000 non-null   int64
31  Vehicle_Risk                           8000 non-null   int64
32  Driver_Age_Risk                        8000 non-null   int32
33  Experience_Risk                        8000 non-null   int64
34  Driver_Risk                            8000 non-null   int64
35  Accident_Month                         8000 non-null   int32
36  Accident_Year                         8000 non-null   int32
37  Accident_Season_Num                   8000 non-null   int32
38  Season                                 8000 non-null   object
39  Accident_Season_Risk                   8000 non-null   int64
40  Claim_Type_Fire                        8000 non-null   bool
41  Claim_Type_Other                       8000 non-null   bool
42  Claim_Type_Theft                       8000 non-null   bool
43  Claim_Type_Vandalism                   8000 non-null   bool
44  Claim_Type_Weather                     8000 non-null   bool
45  Gender_Code                           8000 non-null   int64
dtypes: int64(4), bool(7), category(7), datetime64[ns](3), float64(8), int32(4), int64(9), object(4)
memory usage: 2.0+ MB
```

```
.dataframe tbody tr th {
    vertical-align: top;
}
```

```
.dataframe thead th {
  text-align: right;
}
```

	Claim_ID	Policy_ID	Accident_Date	FNOL_Date	Claim_Complexity	Fraud_Flag	Litigation_Flag	Estimated_Claim_Amount	Ultimate_Claim
0	CLM30000	POL14506	2019-12-19	2019-12-19	Medium	False	True	5243	2808.0
1	CLM30001	POL14338	2018-12-30	2018-12-31	Low	False	False	3934	2952.0
2	CLM30002	POL13575	2021-10-19	2021-10-19	Medium	False	False	153631	156497.0
3	CLM30003	POL10138	2021-06-18	2021-06-18	Low	False	False	2812	1450.0
4	CLM30004	POL12316	2021-03-21	2021-03-24	Low	False	False	5094	4243.0

5 rows × 46 columns

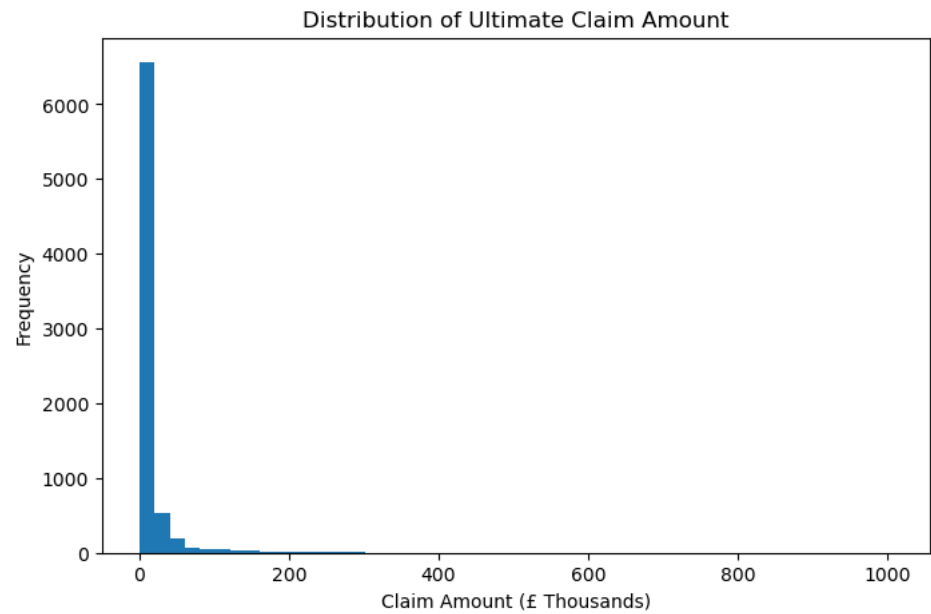
Exploratory Data Analysis(EDA)

```
df.shape
```

```
(8000, 46)
```

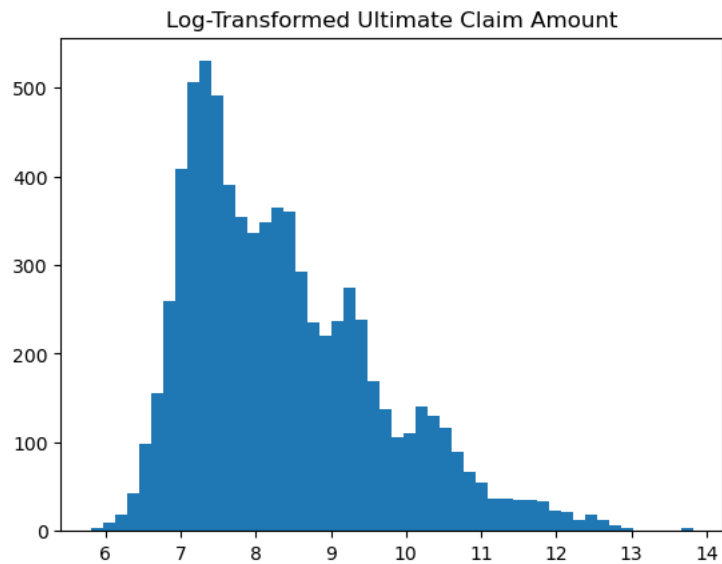
```
# Distribution of Ultimate Claim Amount (Original)
# The histogram shows the raw distribution of claim costs.
# Most claims are relatively small, while a few extreme claims create a long right tail.
# highlighting the skewness and presence of large outliers in the dataset.

plt.figure(figsize=(8,5))
plt.hist(df["Ultimate_Claim_Amount"].dropna()/1000, bins=50, color="#1f77b4")
plt.title("Distribution of Ultimate Claim Amount")
plt.xlabel("Claim Amount (£ Thousands)")
plt.ylabel("Frequency")
plt.show()
```



```
# Distribution of Ultimate Claim Amount (Log-Transformed)**
# The histogram displays the log-transformed claim costs, which compresses extreme values and reduces right skew.
# This makes the distribution more symmetric, highlighting the general pattern of claims while controlling the influence of very large outliers.

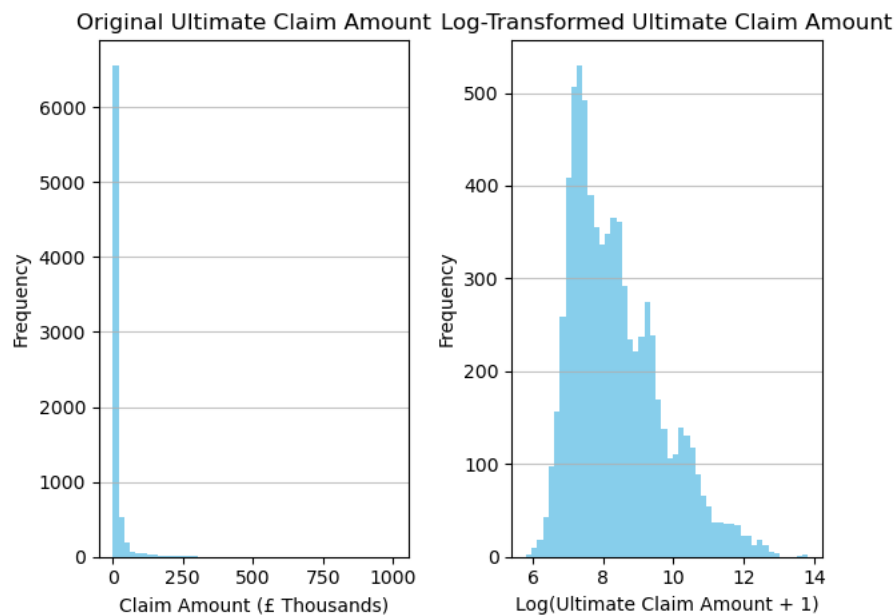
plt.hist(df["Ultimate_Claim_Amount_Log"].dropna(), bins=50)
plt.title("Log-Transformed Ultimate Claim Amount")
plt.show()
```

```
# Original Claim Amount Distribution
plt.subplot(1, 2, 1)
plt.hist(df["Ultimate_Claim_Amount"].dropna()/1000, bins=50, color="skyblue")
plt.title("Original Ultimate Claim Amount")
plt.xlabel("Claim Amount (£ Thousands)")
plt.ylabel("Frequency")
plt.grid(axis='y', alpha=0.75)

# Log-Transformed Claim Amount Distribution
plt.subplot(1, 2, 2)
plt.hist(df["Ultimate_Claim_Amount_Log"].dropna(), bins=50, color="skyblue")
plt.title("Log-Transformed Ultimate Claim Amount")
plt.xlabel("Log(Ultimate Claim Amount + 1)")
plt.ylabel("Frequency")
plt.grid(axis='y', alpha=0.75)

plt.tight_layout()
plt.show()
```



```
df["Vehicle_Type"].value_counts()
```

```
Vehicle_Type
Sedan      2478
Hatchback  1876
Suv        1681
```

```
Coupe      844
Van        720
Motorcycle 401
Name: count, dtype: int64
```

```
df["Season"].value_counts()
```

```
Season
Spring    2091
Fall       2014
Winter     1962
Summer     1933
Name: count, dtype: int64
```

```
df["Severity_Band"].value_counts()
```

```
Severity_Band
Minor          3989
Moderate       1992
Major          1241
Severe          619
Catastrophic    159
Name: count, dtype: int64
```

```
df["Gender_Code"].value_counts()
```

```
Gender_Code
1      4016
0      3984
Name: count, dtype: int64
```

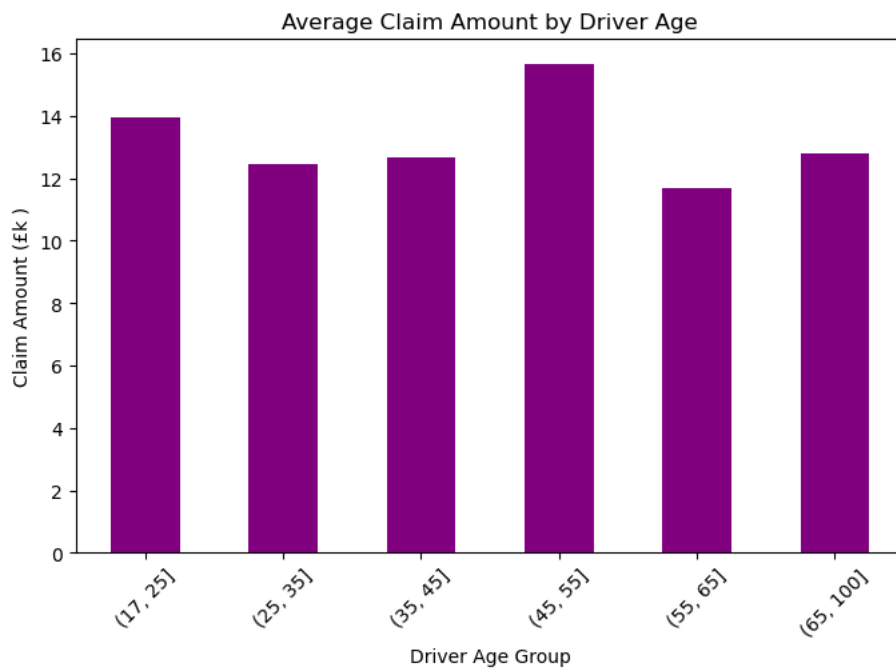
Explore how claim cost varies by key features

Driver Age and Claim Cost Patterns

```
# Bin driver ages for better visualization
df["Driver_Age_Bin"] = pd.cut(df["Age_of_Driver"], bins=[17,25,35,45,55,65,100])

# Aggregate mean claim cost by age bin
driver_age_cost = df.groupby("Driver_Age_Bin", observed=True)["Ultimate_Claim_Amount"].mean() / 1000

# Bar chart
plt.figure(figsize=(8,5))
driver_age_cost.plot(kind="bar", color="purple")
plt.title("Average Claim Amount by Driver Age")
plt.ylabel("Claim Amount (£k)")
plt.xlabel("Driver Age Group")
plt.xticks(rotation=45)
plt.show()
```

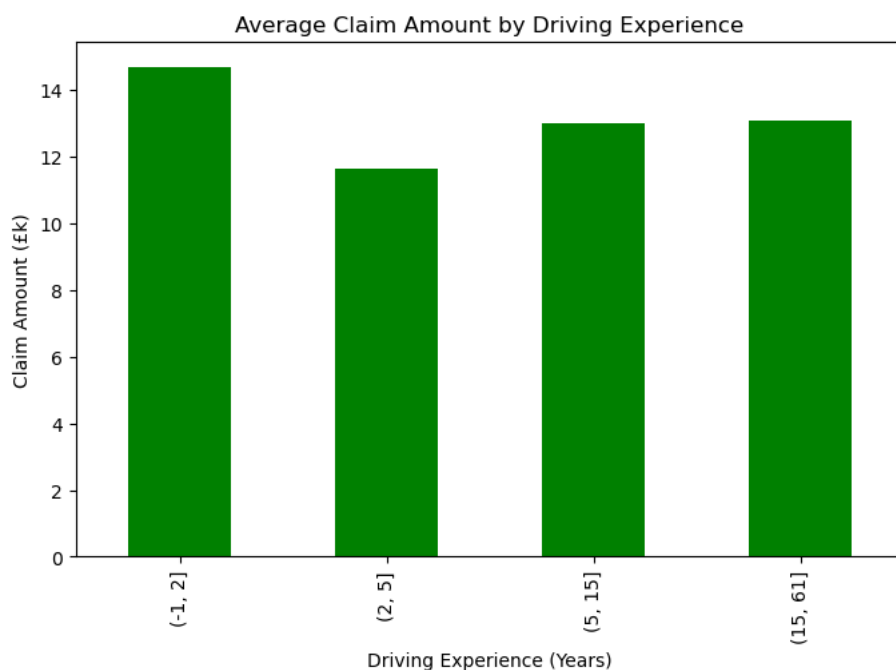


Driving Experience and Claim Cost Patterns

```
df["Driving_Experience_Bin"] = pd.cut(df["Driving_Experience_Years"], bins=[-1,2,5,15,61])

experience_cost = df.groupby("Driving_Experience_Bin", observed=True)["Ultimate_Claim_Amount"].mean()/1000

plt.figure(figsize=(8,5))
experience_cost.plot(kind="bar", color="green")
plt.title("Average Claim Amount by Driving Experience")
plt.ylabel("Claim Amount (£k)")
plt.xlabel("Driving Experience (Years)")
plt.show()
```



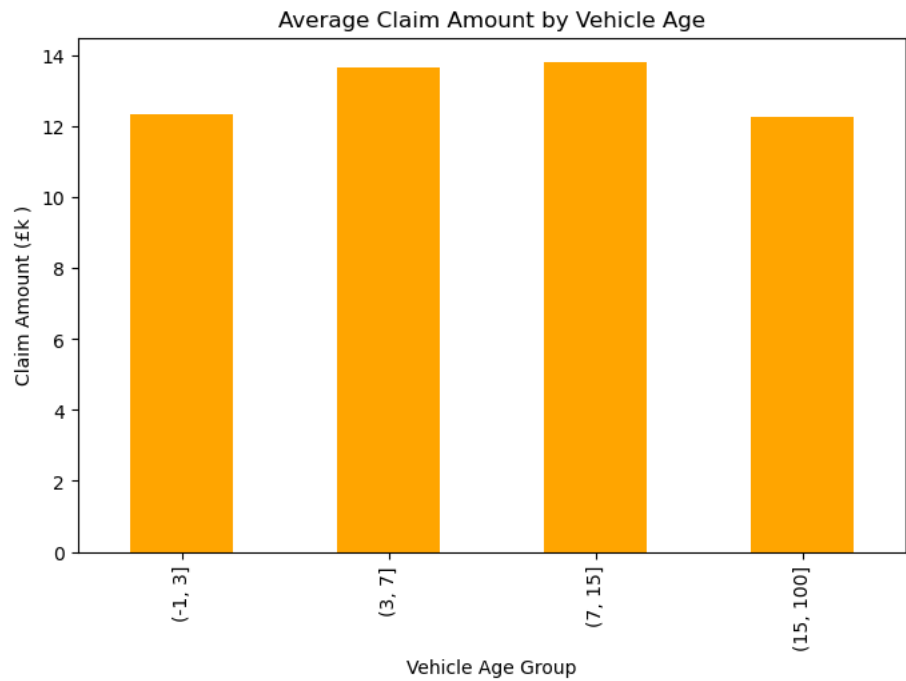
Vehicle Age and Claim Cost Patterns

```
# Bin vehicle ages
df["Vehicle_Age_Bin"] = pd.cut(df["Vehicle_Age"], bins=[-1,3,7,15,100])

vehicle_age_cost = df.groupby("Vehicle_Age_Bin", observed=True)["Ultimate_Claim_Amount"].mean()/1000

plt.figure(figsize=(8,5))
```

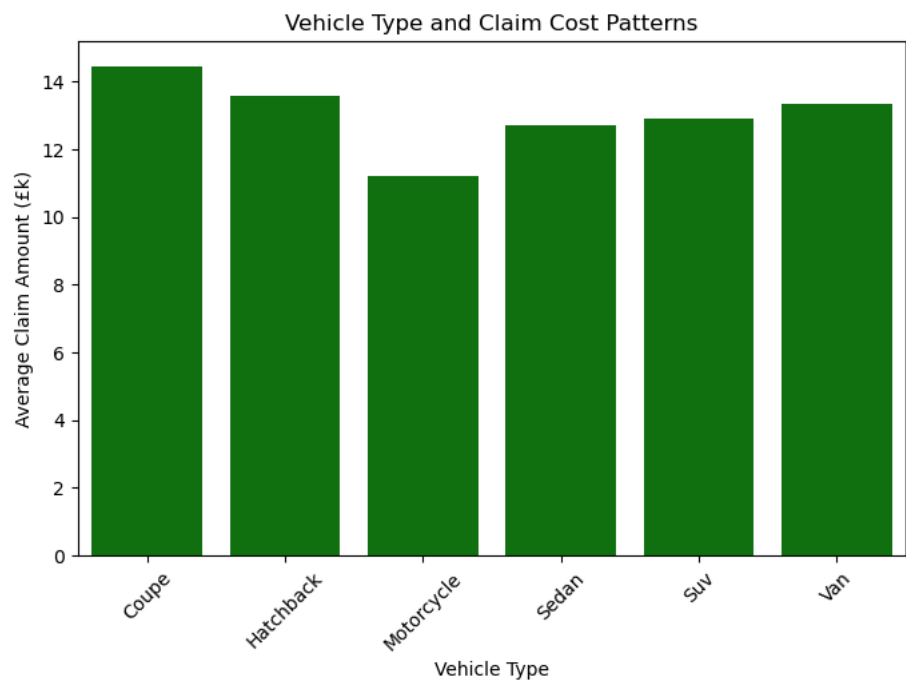
```
vehicle_age_cost.plot(kind="bar", color="orange")
plt.title("Average Claim Amount by Vehicle Age")
plt.ylabel("Claim Amount (£k)")
plt.xlabel("Vehicle Age Group")
plt.show()
```



Vehicle Type and Claim Cost Patterns

```
# Average claim amount by vehicle type (in £k)
vehicle_claim_cost = df.groupby("Vehicle_Type", observed=True)["Ultimate_Claim_Amount"].mean().sort_values(ascending=False)
vehicle_claim_cost_k = vehicle_claim_cost/1000 # convert to thousands

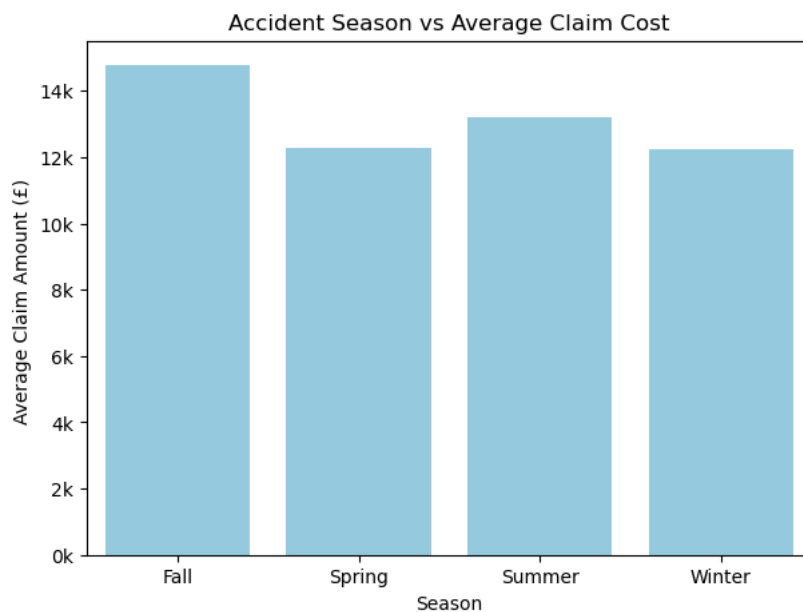
# Plot
plt.figure(figsize=(8,5))
sns.barplot(x=vehicle_claim_cost_k.index,y=vehicle_claim_cost_k.values,color="green")
plt.title("Vehicle Type and Claim Cost Patterns")
plt.xlabel("Vehicle Type")
plt.ylabel("Average Claim Amount (£k)")
plt.xticks(rotation=45)
plt.show()
```



Accident Season and Claim Cost Patterns

```
# Average Ultimate Claim Amount per season (actual £)
season_claim_cost = df.groupby("Season")["Ultimate_Claim_Amount"].mean().reset_index()

plt.figure(figsize=(7,5))
sns.barplot(
    x="Season",
    y="Ultimate_Claim_Amount",
    data=season_claim_cost,
    color="skyblue"
)
plt.title("Accident Season vs Average Claim Cost")
plt.ylabel("Average Claim Amount (£) ")
plt.xlabel("Season")
plt.gca().yaxis.set_major_formatter(plt.FuncFormatter(lambda x, _: f'{int(x/1000)}k'))
plt.show()
```

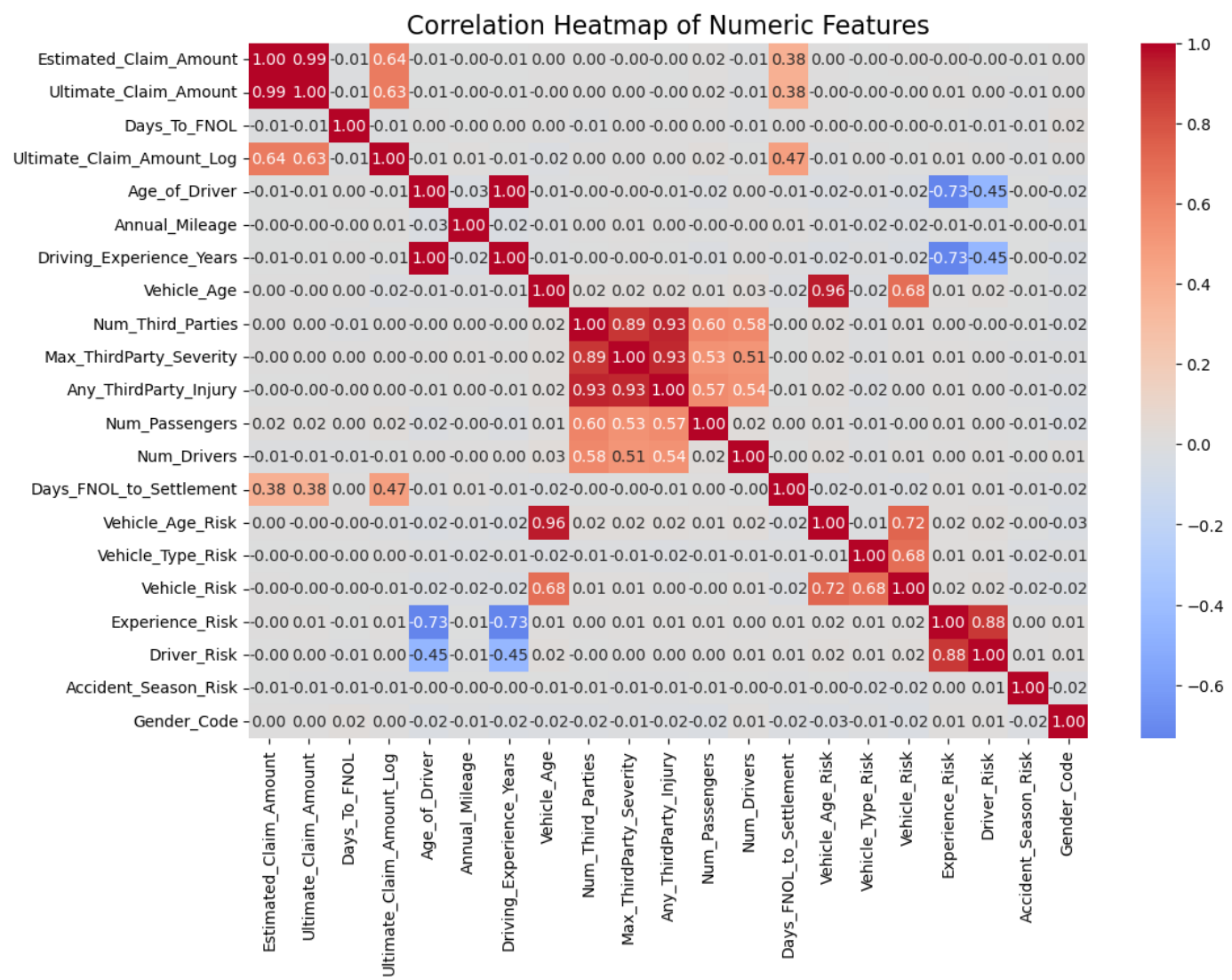


Correlation of Numeric Features

```
# Select only numeric columns
numeric_cols = df.select_dtypes(include=["int64", "float64"])

# Compute correlation matrix
corr_matrix = numeric_cols.corr()

# Plot heatmap
plt.figure(figsize=(12, 8))
sns.heatmap(
    corr_matrix,
    annot=True,          # show correlation values
    fmt=".2f",           # 2 decimal places
    cmap="coolwarm",     # red-blue gradient
    center=0             # center at 0
)
plt.title("Correlation Heatmap of Numeric Features", fontsize=16)
plt.show()
```



Heatmap Interpretation

The heatmap shows that the Estimated Claim Amount has an extremely strong positive correlation (≈ 0.99) with the Ultimate Claim Amount, indicating it is a key predictor of final claim cost. Settlement duration also shows a moderate positive relationship, suggesting that longer and more complex claims tend to result in higher payouts.

Driver demographics, vehicle age, and seasonality exhibit weak linear correlations with claim cost, implying that their impact is likely non-linear or interaction-based rather than direct. Engineered vehicle risk features show strong internal correlations, validating the feature construction but also highlighting potential multicollinearity for linear models. Overall, the heatmap guides feature selection while emphasizing the need for non-linear models to fully capture claim cost drivers.

The correlation heatmap is used as an exploratory tool to screen FNOL and policy variables for linear relationships with claim cost. While variables such as estimated claim amount and settlement duration show strong direct associations, several policy and driver-related features exhibit weak linear correlations, suggesting that their influence may be non-linear or interaction-based. These variables are therefore retained for further evaluation using machine learning models.

```
df.to_csv("final_engineered_claims_dataset.csv", index=False)
print("Final engineered dataset saved successfully.")
```

Final engineered dataset saved successfully.